Syracuse University
School of Information Studies

Week 2
Introduction to Data Science

Dr. Preeti Jagadev

Data Frames in R

Rows and Columns:

- Think of a table. Here's an example of a small table:

Name	Age	Grade
Riya	12	A
Aryan	13	B
Zoya	12	A+

- In this table: Each row (**Riya, Aryan, Zoya**) is a case or **instance.**
- Each column (**Name, Age, Grade**) is a variable or **attribute.**

Data Frame in R:

- In R, this kind of table is stored in a special object called a data frame.

Parts of a Data Frame:

When you're working with a data frame, you want to understand:

- **Context:** What is the data about?
  - Example: This dataset is about students and their academic performance.

- **Content:** What is in the columns?
  - **Example:** "Name", "Age", and "Grade" are columns.

- **Mode/Type**: What kind of data is in each column?
  - **Example:** Names (Riya, Aryan),
  - Numbers (numeric or integer): e.g., Age (12, 13)
  - Categories (factor): e.g., Grade (A, B, A+)

Data Frames Make Analysis Easy in R:

- Because data is neatly arranged: You can filter, sort, summarize, or plot data easily.
- R has many functions that work well with data frames.

Features of Data Frames in R:

Let’s understand some key features:

- **Enforcement of Rectangle:**
  - Every column must have the same number of rows.
  - You can’t have one column with 3 entries and another with 5.
  - That would break the data frame.

- **Row and Column Naming:**
  - Columns usually have names: “Name”, “Age”, etc.
  - Rows can also have names (but usually are just numbered by default: 1, 2, 3…

- **Single Type by Column:**
  - Each column must have one type of data.
  - You can't mix text and numbers in one column.
  - So, a column is either all numbers, or all text, or all dates, etc.

Unique Identifier (Case Label):

- In a well-organized dataset (or data frame in R): Each row should represent one unique thing (person, item, event, etc.)
- To tell them apart, we usually give each row a unique identifier, like a name tag.
- This identifier is often called a case label or ID.
- **Example:**

Student_ID	Name	Age	Grade
1001	Riya	12	A
1002	Aryan	13	B
1003	Zoya	12	A+

- Here, Student_ID is a unique identifier.No two students have the same ID.
- This helps us refer to any student easily and uniquely.

R Does NOT Enforce Uniqueness:

- Unlike some databases (like SQL), R Does not check whether each row is unique.
- You can have two or more identical rows, R won't complain.

Tibbles vs. Data Frames

What is a Tibble?

- A Tibble is a modern version of a data frame.
- **Simple Analogy:** Think of a data frame like an older model car. It works, but it doesn't have modern features. A Tibble is like a newer car with a touch screen, backup camera, and better fuel efficiency, still a car, but smarter and more comfortable to use.
- Tibbles are smarter and more user-friendly, especially when you're working with data in the tidyverse (collection of R packages designed for data science).
- **No automatic conversions:** In regular data frames, R sometimes changes data types without asking (e.g., turning strings into factors). Tibbles don't do that, they keep the data as you gave it.
- When exploring a tibble, you can use the ***glimpse()*** function to quickly view its structure.
- It gives a cleaner, horizontal summary of the data compared to ***str().***

Tibble Output:

- The light blue block shows what a Tibble looks like when printed.
- It's telling us: This table has 1,000 rows and 5 columns.
- The column names are a, b, c, d, and e.
- Under each column name, it shows the type of data in that column (like <dttm>, <int>, etc.).
- It prints the first few rows and then says there's more (not printing all 1,000 rows at once, which is helpful).

```
#> # A tibble: 1,000 x 5
#>   a                   b              c     d e
#>   <dttm>              <date>     <int> <dbl> <chr
#> 1 2019-01-08 17:33:11 2019-01-15     1 0.368 h
#> 2 2019-01-09 11:38:20 2019-01-20     2 0.612 n
#> 3 2019-01-09 06:02:00 2019-01-30     3 0.415 l
#> 4 2019-01-08 19:23:17 2019-01-29     4 0.212 x
#> 5 2019-01-08 15:47:33 2019-01-26     5 0.733 a
#> 6 2019-01-09 02:48:30 2019-01-22     6 0.460 v
#> # … with 994 more rows
```

Label	Type	Description
<int>	Integer	Whole numbers: e.g., 1, 42, 2025
<dbl>	Double (Numeric)	Decimal numbers: e.g., 3.14, 0.75, -10.5
<chr>	Character	Text : e.g., "apple", "John", "R is fun"
<lgl>	Logical	TRUE or FALSE values
<dttm>	Date-Time	Date and time e.g., 2023-12-31 14:30:00
<date>	Date	Just the date : e.g., 2023-12-31
<fct>	Factor	Categorical : e.g., "small", "medium", "large"

Coding with R

Summary()

Gives a quick overview of the statistical properties (like **min, median, mean, and max**) for the numerical columns in a dataframe.

Creating a Dataframe in R

```
myFamilyNames <- c("Dad", "Mom", "Sis", "Bro", "Dog")
myFamilyNames
```

```
## [1] "Dad" "Mom" "Sis" "Bro" "Dog"
```

```
myFamilyAges <- c(43, 42, 12, 8, 5)
myFamilyAges
```

```
## [1] 43 42 12  8  5
```

```
myFamilyGenders <- c("Male", "Female", "Female", "Male", "Female")
myFamilyGenders
```

```
## [1] "Male"   "Female" "Female" "Male"   "Female"
```

```
myFamilyWeights <- c(188, 136, 83, 61, 44)
myFamilyWeights
```

```
## [1] 188 136  83  61  44
```

```
myFamily <- data.frame(myFamilyNames, myFamilyAges, myFamilyGenders, myFamilyWeights)
myFamily
```

```
##   myFamilyNames myFamilyAges myFamilyGenders myFamilyWeights
## 1           Dad           43            Male             188
## 2           Mom           42          Female             136
## 3           Sis           12          Female              83
## 4           Bro            8            Male              61
## 5           Dog            5          Female              44
```

```r
str(myFamily)
```

```
## 'data.frame':    5 obs. of  4 variables:
##  $ myFamilyNames  : chr  "Dad" "Mom" "Sis" "Bro" ...
##  $ myFamilyAges   : num  43 42 12 8 5
##  $ myFamilyGenders: chr  "Male" "Female" "Female" "Male" ...
##  $ myFamilyWeights: num  188 136 83 61 44
```

```r
summary(myFamily)
```

```
##  myFamilyNames       myFamilyAges myFamilyGenders    myFamilyWeights
##  Length:5           Min.   : 5    Length:5           Min.   : 44.0
##  Class :character   1st Qu.: 8    Class :character   1st Qu.: 61.0
##  Mode  :character   Median :12    Mode  :character   Median : 83.0
##                     Mean   :22                       Mean   :102.4
##                     3rd Qu.:42                       3rd Qu.:136.0
##                     Max.   :43                       Max.   :188.0
```

This is a list of 5 names, and all values are text.

Min.: 5 (The youngest family member is 5 years old).
1st Qu.: 8 (25% of the ages are below or equal to 8.
Median: 12 (The middle age (when sorted) is 12).
Mean: 22 (The average age of myfamily is 22 years).
3rd Qu.: 42 (75% of the ages are below or equal to 42).
Max.: 43 (The oldest person is 43 years old).

Accessing Dataframes

Accessing Dataframes as a vector:

```
myFamily$myFamilyAges[2]
```

```
## [1] 42
```

```
#Returns the second element in myFamilyAges
```

Accessing Dataframes as a matrix:

```
myFamily[1,1]
```

```
## [1] "Dad"
```

```
#Returns the data element in the first row and first column
```

```r
myFamily[1,]
```

```
##   myFamilyNames myFamilyAges myFamilyGenders myFamilyWeights
## 1           Dad           43             Male              188
```

```r
#Returns the first row
```

```r
myFamily[,1]
```

```
## [1] "Dad" "Mom" "Sis" "Bro" "Dog"
```

```r
#Returns the first column
```

```r
myFamily[-1,]
```

```
##   myFamilyNames myFamilyAges myFamilyGenders myFamilyWeights
## 2           Mom           42          Female              136
## 3           Sis           12          Female               83
## 4           Bro            8            Male               61
## 5           Dog            5          Female               44
```

```r
#Returns everything but the 1st row  (i.e., deletes the 1st  row)
```

```r
myFamily[,-1]
```

```
##   myFamilyAges myFamilyGenders myFamilyWeights
## 1           43            Male              188
## 2           42          Female              136
## 3           12          Female               83
## 4            8            Male               61
## 5            5          Female               44
```

```r
#Returns everything but the 1st column (i.e., deletes the 1st column)
```

How to Sort a Data Frame Correctly

- "**mtcars**" is a built-in dataset in R.
- It's a table (data frame) where: **Rows** are different car models, and **Columns** are different measurements (like miles per gallon, horsepower, etc.)
- If you want to sort the "mtcars" data frame, you need to: Pick a specific column to sort by.

For example, suppose you want to sort by the miles per gallon (mpg) column. You do it like

```
mtcars[order(mtcars$mpg), ]
```

```
##                      mpg cyl  disp  hp drat    wt  qsec vs am gear carb
## Cadillac Fleetwood  10.4   8 472.0 205 2.93 5.250 17.98  0  0    3    4
## Lincoln Continental 10.4   8 460.0 215 3.00 5.424 17.82  0  0    3    4
## Camaro Z28          13.3   8 350.0 245 3.73 3.840 15.41  0  0    3    4
## Duster 360          14.3   8 360.0 245 3.21 3.570 15.84  0  0    3    4
## Chrysler Imperial   14.7   8 440.0 230 3.23 5.345 17.42  0  0    3    4
## Maserati Bora       15.0   8 301.0 335 3.54 3.570 14.60  0  1    5    8
## Merc 450SLC         15.2   8 275.8 180 3.07 3.780 18.00  0  0    3    3
## AMC Javelin         15.2   8 304.0 150 3.15 3.435 17.30  0  0    3    2
## Dodge Challenger    15.5   8 318.0 150 2.76 3.520 16.87  0  0    3    2
## Ford Pantera L      15.8   8 351.0 264 4.22 3.170 14.50  0  1    5    4
## Merc 450SE          16.4   8 275.8 180 3.07 4.070 17.40  0  0    3    3
## Merc 450SL          17.3   8 275.8 180 3.07 3.730 17.60  0  0    3    3
## Merc 280C           17.8   6 167.6 123 3.92 3.440 18.90  1  0    4    4
## Valiant             18.1   6 225.0 105 2.76 3.460 20.22  1  0    3    1
## Hornet Sportabout   18.7   8 360.0 175 3.15 3.440 17.02  0  0    3    2
## Merc 280            19.2   6 167.6 123 3.92 3.440 18.30  1  0    4    4
## Pontiac Firebird    19.2   8 400.0 175 3.08 3.845 17.05  0  0    3    2
## Ferrari Dino        19.7   6 145.0 175 3.62 2.770 15.50  0  1    5    6
## Mazda RX4           21.0   6 160.0 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag       21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
## Hornet 4 Drive      21.4   6 258.0 110 3.08 3.215 19.44  1  0    3    1
## Volvo 142E          21.4   4 121.0 109 4.11 2.780 18.60  1  1    4    2
## Toyota Corona       21.5   4 120.1  97 3.70 2.465 20.01  1  0    3    1
## Datsun 710          22.8   4 108.0  93 3.85 2.320 18.61  1  1    4    1
## Merc 230            22.8   4 140.8  95 3.92 3.150 22.90  1  0    4    2
```

If you want descending order, use:

```
mtcars[order(-mtcars$mpg), ]
```

```r
indexes <- order(mtcars[,"mpg"])
```

In the above chunk: "mtcars[,"mpg"]" extracts the "mpg" column (miles per gallon).

"order(...)": Returns the row numbers that would sort the data in increasing order of "mpg".

You save those row numbers in indexes.

Look at the first index

```r
indexes[1]
```

```
## [1] 15
```

This tells you: the 15th row of "mtcars" has the lowest "mpg" (i.e., it is the least fuel-efficient car).

Look at that row

```r
mtcars[15,]
```

```
##                     mpg cyl disp  hp drat   wt  qsec vs am gear carb
## Cadillac Fleetwood 10.4   8  472 205 2.93 5.25 17.98  0  0    3    4
```

This shows the entire 15th row of the "mtcars" data frame.This car has an "mpg" of 10.4, which is very low — meaning it consumes a lot of fuel.

Exploring the dataframe the tidyverse way

```r
install.packages("tidyverse")
```

```
## Installing package into '/cloud/lib/x86_64-pc-linux-gnu-library/4.4'
## (as 'lib' is unspecified)
```

```r
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.2     v tibble    3.2.1
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.4
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become error
```

```r
glimpse(mtcars)
```

```
## Rows: 32
## Columns: 11
## $ mpg  <dbl> 21.0, 21.0, 22.8, 21.4, 18.7, 18.1, 14.3, 24.4, 22.8, 19.2, 17.8,~
## $ cyl  <dbl> 6, 6, 4, 6, 8, 6, 8, 4, 4, 6, 6, 8, 8, 8, 8, 8, 8, 4, 4, 4, 4, 8,~
## $ disp <dbl> 160.0, 160.0, 108.0, 258.0, 360.0, 225.0, 360.0, 146.7, 140.8, 16~
```

Selecting a Column the tidyverse way

"select()" lets you choose specific columns from a dataset (like a spreadsheet).

It returns a smaller version of the dataset with only the columns you asked for.

```r
select(mtcars, mpg)
```

Selecting more than one column

```
mtcars[ , c("mpg", "hp")]
```

```
##                     mpg  hp
## Mazda RX4          21.0 110
## Mazda RX4 Wag      21.0 110
## Datsun 710         22.8  93
## Hornet 4 Drive     21.4 110
## Hornet Sportabout  18.7 175
## Valiant            18.1 105
## Duster 360         14.3 245
## Merc 240D          24.4  62
## Merc 230           22.8  95
## Merc 280           19.2 123
## Merc 280C          17.8 123
## Merc 450SE         16.4 180
## Merc 450SL         17.3 180
## Merc 450SLC        15.2 180
## Cadillac Fleetwood 10.4 205
```

Selecting more than one column, the tidyverse way:

```
select(mtcars, mpg, hp)
```

```
##                     mpg  hp
## Mazda RX4          21.0 110
## Mazda RX4 Wag      21.0 110
## Datsun 710         22.8  93
## Hornet 4 Drive     21.4 110
## Hornet Sportabout  18.7 175
## Valiant            18.1 105
## Duster 360         14.3 245
## Merc 240D          24.4  62
## Merc 230           22.8  95
## Merc 280           19.2 123
## Merc 280C          17.8 123
## Merc 450SE         16.4 180
## Merc 450SL         17.3 180
## Merc 450SLC        15.2 180
## Cadillac Fleetwood 10.4 205
## Lincoln Continental 10.4 215
## Chrysler Imperial  14.7 230
## Fiat 128           32.4  66
## Honda Civic        30.4  52
## Toyota Corolla     33.9  65
## Toyota Corona      21.5  97
```

Pipe Operator

Instead of nesting functions inside each other, you can write it using a pipe:

```
x <- c(4, 1, 10, 5)
x %>% min
```

```
## [1] 1
```

This means:

"Take x, then find the minimum."

How It Works:

The pipe takes the result of the left-hand side (x) and passes it as the first argument to the function on the right (min()).

So:

x %>% min() is the same as: min(x)

Pipes make code: Cleaner,and easier to read.

```
mtcars %>% slice(1:2)
```

"Take the mtcars dataset, then give me the first two rows."

Question

```
names <- c("Riya", "Omar", "Lina")
ages <- c(21, 25, 23)
emails <- c("riya@syr.edu", "omar@syr.edu", "lina@syr.edu")
students <- data.frame(names, ages, emails)
```

Que 1. After running the above commands to declare a data frame named students, **which statement would you use to print Omar's email?**

2 → second row (Omar)

3 → third column (emails)

So, students[2,3]

	Names	Ages	Emails
1	Riya	21	riya@syr.edu
2	Omar	25	omar@syr.edu
3	Lina	23	lina@syr.edu

Question

```
names <- c("Riya", "Omar", "Lina")
ages <- c(21, 25, 23)
emails <- c("riya@syr.edu", "omar@syr.edu", "lina@syr.edu")
students <- data.frame(names, ages, emails)
```

Que 2. After running the above commands to declare a data frame named students, which statement prints **Lina's age?**

3 → third row (Lina)

2 → second column (ages)

So, Students[3,2]

	Names	Ages	Emails
1	Riya	21	riya@syr.edu
2	Omar	25	omar@syr.edu
3	Lina	23	lina@syr.edu

Weekly Quiz

Syracuse University
School of Information Studies

The End